

L01 Introduction & History

What is DL • taxonomy • shallow vs deep

DL = ML methods using **neural networks** to learn **representations from raw data**. (从原始数据学表示)
Taxonomy AI $\supset$ ML $\supset$ Deep Learning $\supset$ DL.

Two problems: discriminative $X \rightarrow Y$ (classification); **generative** $Y \rightarrow X$ (generation).

Shallow = hand-design fixed features then learn f : $y=f(\phi(x); \theta_f)$, ϕ fixed. **Deep** = learn ϕ and f end-to-end: $y=f(\text{NN}(x; \theta_{NN}); \theta_f)$.

2 assumptions (recur all course): **(1) expressivity** – net approximates **any** fn ($\hookrightarrow$ universal approx); **(2) learnability** – gradient training finds a good net in practice (no optimality guarantee).

Historical neuron / learning models

McCulloch-Pitts ('43): $g = \sum_i x_i, y=1[g > \theta] \Rightarrow$ any Boolean fn, but **no learning rule**.

Hebb ('49, first learning rule, "fire together wire together"): $w_i \leftarrow w_i + \eta x_i y_i$ (unsupervised, correlational).

Perceptron (Rosenblatt '58)

Output $y=1[\sum_i w_i x_i - T > 0]$. **Rule** (d =desired) $\Rightarrow$ update only when wrong:

$$w \leftarrow w + \eta (d(x) - y) x$$

Convergence: provably converges **iff data linearly separable** – each mistake nudges w toward the correct half-plane, mistake count finite. Mimics any single gate AND/OR/NOT.

XOR killer (Minsky&Papert '69): 1 perceptron **cannot** do $X \oplus Y$ (not lin-separable). **Fix = MLP**: hidden layer builds $X \oplus Y = (X \vee Y) \wedge (X \wedge Y)$ (OR + NAND into hidden, AND at output).

Why deep (expressivity)

Geometry: layer-1 = half-planes; hidden combos = **convex regions**; deeper = **arbitrary regions**.

2-layer MLP = universal Boolean (DNF over hidden AND-nodes). **Parity**: N -bit XOR needs $O(2^{N-1})$ units in one hidden layer (one/minterm).

Håstad ('87) parity bound: depth- k circuit size $\exp(\Omega(N^{1/(k-1)})) \Rightarrow$ **depth buys exponential savings over width**.

***Universal approx** (Cybenko/Hornik): 1 hidden layer of sigmoids $\Rightarrow$ any continuous fn on a compact set.

Hardness: training a **3-node net is NP-complete** (Blum-Rivest '92) $\Rightarrow$ exact optimal training intractable. AI-winter takeaway: MLP is a **universal approximator** but no algorithm **finds** the optimum (expressive yet untrainable).

The turn: sigmoid $\rightarrow$ backprop

Why hard thresholds fail: 0/1 output $\Rightarrow$ large weight changes leave error flat $\Rightarrow$ **no gradient** $\Rightarrow$ combinatorial opt.

Fix $\leftarrow$ differentiable: $z = \sum_i w_i x_i$,

$$y = \sigma(z) = \frac{1}{1 + e^{-z}} = P(Y=1|X).$$

*derivative $\sigma'(z) = \sigma(1 - \sigma(z))$ (used in all over backprop).

Backprop (Rumelhart-Hinton-Williams '86; Werbos '74): back-propagate gradients via chain rule $\Rightarrow$ first to **automatically learn representations**; still no optimality guarantee.

History: CNN LeCun '89 $\rightarrow$ LeNet'98; neural-LM Bengio'03; attention Bahdanau'15; AlexNet'12 (DL era); GPT-3'20, AlphaFold'20, ChatGPT'22, o1'24. Turing'18: Hinton: LeCun-Bengio.

perceptron rule + linear-separability convergence; **why XOR needs a hidden layer**.

hard-threshold (no gradient) $\rightarrow$ sigmoid $\rightarrow$ backprop; $\sigma' = \sigma(1 - \sigma)$.

why-deep: 2-layer universal Boolean, parity $O(2^{N-1})$, Håstad depth bound.

L02 Supervised Learning I – Classification • backprop • CNN

Framing & ERM

DL idea: replace hand-tuned feature ϕ by a differentiable net: $y=f(\phi(x); \theta) \rightarrow y=f_{NN}(x; \theta)$ – the net **learns features** (representation learning); the "algorithm" = architecture + weights.

Population obj. (binary $y \in \{0, 1\}$):

$$\theta^* = \arg \min_{\theta} \int \text{err}(f(x; \theta), y) P(x) dx.$$

ERM: replace integral by sample mean:

$$\hat{\theta} = \arg \min_{\theta} \frac{1}{n} \sum_{i=1}^n \text{err}(f(x_i; \theta), y_i).$$

Logistic / sigmoid neuron (binary)

Model: $P(y=1) = \sigma(w^T x + b)$, $\sigma(z) = \frac{1}{1 + e^{-z}}$. Single-layer perceptron = one sigmoid neuron.

Loss (NLL/CE): $-\log \sigma$ if $y=1$, $-\log(1 - \sigma)$ if $y=0$; compact $-\log \log p(1-y) \log(1-p)$, $p = \sigma(w^T x + b)$.

Deriv $\sigma' = \sigma(1 - \sigma)$: let $u=1+e^{-z}$, $\sigma = u^{-1}$.

$$\sigma'(z) = -u^{-2} \cdot u' = -u^{-2} (e^{-z}) = \frac{e^{-z}}{(1+e^{-z})^2} = \frac{\sigma}{1+\sigma} \cdot \frac{1-\sigma}{1+\sigma} = \sigma(1-\sigma)$$

Deriv grad $= (\sigma - y)x$ (case $y=1$, loss $E = -\log \sigma$; $z = w^T x$):

$$\nabla_w L = -\frac{\partial}{\partial z} \nabla_w \sigma = -\frac{\partial}{\partial z} \sigma'(z) \nabla_w z = -\frac{\partial}{\partial z} \sigma(1 - \sigma) x = -(1 - \sigma) x$$

$$= (\sigma - 1)x = \frac{e^z}{1+e^z} x = (\sigma - y)x$$

Case $y=0$ ($\ell = -\log(1 - \sigma)$): $\nabla_w L = \frac{\partial}{\partial z} \sigma(1 - \sigma) x = \sigma x = \sigma(0 - 1)x$.

Both cases $\Rightarrow \left[\frac{\nabla \text{err}}{\partial z} = (\sigma - y)x \right]$ – (pred-label) $\times$ input. Also $\nabla_{b,c} L = w$.

DL is generally non-convex; if L convex $\Rightarrow$ global opt reachable, else no guarantee.

Optimization recap

Recipe: solve $\nabla L = 0$, verify min via Hessian $[\nabla^2 L]_{ij} = \frac{\partial^2 L}{\partial \theta_i \partial \theta_j}$. PD. Analytic infeasible $\Rightarrow$ iterate.

Gradient = direction of fastest increase, $\perp$ level curves; $df = \nabla f \cdot dX$.

GD: $X_{k+1} = X_k - \eta_k \nabla f(X_k)$, stop when $\|f(X_{k+1}) - f(X_k)\| < \epsilon$. May hit local min/saddle.

LR: η : small=safe&slow, large=diverge/zig-zag. Ex $x_1^2 + x_2^2 + 4x_2$, $X_0 = (3, 3)$; $\eta=1$ smooth, $\eta=2$ oscillates.

Softmax + cross-entropy (multi-class)

Need: a distribution over C classes ($f_i \geq 0, \sum_i f_i = 1$); sigmoid only gives binary scalar.

Softmax: logits $z = Wx$ ($W \in \mathbb{R}^{(d+1) \times C}$, bias row),

$$z_c = \sum_i w_{ci} x_i; f_c = \text{softmax}(z) = \frac{e^{z_c}}{\sum_i e^{z_i}}$$

Loss $-\log f_c = -(z_c - \log \sum_i e^{z_i})$ (log-sum-exp form). = $CE(p, q) = -\sum_i p_i \log q_i$ with one-hot p .

Deriv $\partial L / \partial z_c = f_c - 1$ [c=y] ($L = -\log f_c = -z_c + \log \sum_i e^{z_i}$):

$$\begin{aligned} \frac{\partial L}{\partial z_c} &= -\frac{\partial}{\partial z_c} \left(\frac{e^{z_c}}{\sum_i e^{z_i}} \right) + \frac{\partial}{\partial z_c} \log \sum_i e^{z_i} \\ &= -\frac{1}{\sum_i e^{z_i}} \left(e^{z_c} - \sum_i e^{z_i} \right) + \frac{e^{z_c}}{\sum_i e^{z_i}} \\ &= f_c - 1 = [c=y] \end{aligned}$$

Chain to weights: $\frac{\partial L}{\partial w_{ci}} = x_i$, so

$$\frac{\partial L}{\partial w_{ci}} = \frac{1}{N} \sum_j (f_c - 1) [c=y_i] x_j^i = (\text{prob-one-hot}) \otimes x^i$$

Input. (与二分类 $\sigma - y$ 同形)

MLP

N -layer: $y^{(0)} = x$; layer k : $z^{(k)} = W^{(k)} y^{(k-1)} + b^{(k)}$, $y^{(k)} = f_k(z^{(k)})$;

out $y = \text{softmax}(y^{(N)})$. Bias trick: absorb b into W (append constant 1 feature).

Learn: GD on every weight $w_{ij}^{(k)} \leftarrow w_{ij}^{(k)} - \eta \frac{\partial L}{\partial w_{ij}^{(k)}}$ – need grad of every weight $\Rightarrow$ backprop.

Backpropagation

Chain rule: $y = f(g_1(x), \dots, g_M(x)) \Rightarrow \frac{\partial L}{\partial x_i} = \sum_m \frac{\partial L}{\partial g_m} \frac{\partial g_m}{\partial x_i}$.

Forward (save all): set $y^{(0)} = x$, $y_{\text{bias}}^{(k)} = 1$; for $k=1..N$: $z^{(k)} = W^{(k)} y^{(k-1)}$, $y^{(k)} = f_k(z^{(k)})$; output $Y = y^{(N)}$. Keep every $z^{(k)}$, $y^{(k)}$ for reuse.

Backward derivation (reverse order, $D_{\text{rev}} = \text{loss}$):

(out y) $\frac{\partial L}{\partial y^{(N)}}$ direct from loss

$$\text{(thru } f) \quad \frac{\partial L}{\partial z^{(N)}} = \frac{\partial L}{\partial y^{(N)}} \frac{\partial y^{(N)}}{\partial z^{(N)}} = f'_N \frac{\partial L}{\partial y^{(N)}}$$

Weight grad at output: $z_i^{(N)} = \sum_j w_{ij}^{(N)} y_j^{(N-1)} \Rightarrow$

$$\frac{\partial L}{\partial w_{ij}^{(N)}} = \frac{\partial L}{\partial z_i^{(N)}} \frac{\partial z_i^{(N)}}{\partial w_{ij}^{(N)}} = \frac{\partial L}{\partial z_i^{(N)}} y_j^{(N-1)}$$

$$\frac{\partial L}{\partial w_{ij}^{(N)}} = \frac{\partial L}{\partial z_i^{(N)}} \frac{\partial z_i^{(N)}}{\partial y_j^{(N-1)}} \frac{\partial y_j^{(N-1)}}{\partial w_{ij}^{(N)}} = \frac{\partial L}{\partial z_i^{(N)}} \frac{\partial z_i^{(N)}}{\partial y_j^{(N-1)}} \frac{\partial L}{\partial y_j^{(N-1)}}$$

Why weight-grad = input-act $\times$ upstream- δ : a weight enters its pre-activation **linearly**, multiplied by the input activation $\Rightarrow$ local derivative is exactly that activation.

Propagate to prev output: $z_j^{(N)} = \sum_i w_{ji}^{(N)} y_i^{(N-1)} \Rightarrow$

$$\frac{\partial L}{\partial w_{ji}^{(N)}} = \sum_i w_{ji}^{(N)} \frac{\partial L}{\partial z_j^{(N)}} \frac{\partial z_j^{(N)}}{\partial w_{ji}^{(N)}} = \sum_i w_{ji}^{(N)} \frac{\partial L}{\partial z_j^{(N)}} \frac{\partial L}{\partial y_i^{(N-1)}}$$

Thru activation again: $\frac{\partial L}{\partial w_{ji}^{(N)}} = f'_{N-1}(z_i^{(N-1)}) \frac{\partial L}{\partial z_i^{(N-1)}}$; repeat downwards.

Algorithm (memorize), $K = N-1$ to 0:

$$\frac{\partial L}{\partial w_{ij}^{(K)}} = y_j^{(K)} \frac{\partial L}{\partial z_j^{(K)}} \quad (\text{weight})$$

$$\frac{\partial L}{\partial w_{ij}^{(K)}} = \sum_l w_{jl}^{(K+1)} \frac{\partial L}{\partial z_j^{(K)}} \frac{\partial z_j^{(K)}}{\partial w_{ij}^{(K)}} \quad (\text{thru } W)$$

$$\frac{\partial L}{\partial w_{ij}^{(K)}} = \frac{\partial L}{\partial z_j^{(K)}} f'_K(z_j^{(K)}) \quad (\text{thru } f)$$

Cost $\approx$ 1 forward pass (every saved value reused once); these 3 recurrences are the most-tested item.

Activations (f, f')

Sigmoid $\sigma = \frac{1}{1 + e^{-z}}$, $f' = \sigma(1 - \sigma)$ – prob interp, but output always $\rightarrow 0$, saturates $\Rightarrow$ **vanishing grad**.

tanh $\tanh^2 z = 1 - \tanh^2 z$ – zero-centered (fixes sigmoid sign), still saturates.

ReLU $\max(x, 0)$, $f' = 1[x > 0]$ (subgrad $\in [0, 1]$ at 0) – no saturation $\Rightarrow$ OK; "dead" for $x < 0$.

LeakyReLU $\max(ax, x)$, $f' = 1$ or 0; **ELU** $x(x \geq 0) / (e^x - 1)$ ($x < 0$), $f' = 1 + e^x$.

Subgradient = any direction that decreases f ; gradients are subgrads, not vice versa. Quiz

$f(X) = \max_i x_i$, $\nabla f = \frac{1}{2} [x_i = \max_i x_i]$ (ties $\rightarrow$ simplex) – same structure as max-pool backprop.

Width/depth • regularization

Neuron $f(w^T x) =$ matched/correlation filter (fires when x correlates with w); MLP = cascade of features $\Rightarrow$ put more neurons in early layers.

Depth hard: grads are **products over layers** $\Rightarrow$ vanish/explode (motivates ReLU, careful init; next lecture).

L2 / weight decay: $L = \text{Loss} + \alpha \|w\|^2 \Rightarrow \nabla_w L = \nabla \text{Loss} + \alpha w$; step adds $-\alpha w$ $\Rightarrow$ shrinks weights each step.

MLP $\rightarrow$ CNN: shift invariance

Problem: a plain MLP filter trained at one location doesn't transfer $\Rightarrow$ **need shift invariance** (face regardless of pattern position).

Scan solution: one shared local MLP scans every patch; combine over positions by Max (Boolean OR) or a final MLP. Scanning MLP $\ll$ dense MLP params (only the small shared patch net).

Tied-weight grad (weight-sharing constraint): if $S = \{s_1, \dots, s_N\}$ are edges sharing one value,

$$\nabla_{\text{shared}} L = \sum_{i \in S} w_i L_i = \text{sum the per-edge grads (each shared edge contributes once via chain rule)}.$$

Convolution & distributed scanning

Conv (cross-corr): stack feature maps; next layer reads the same position across maps. Accumulation

$$z(i, j, x, y) = \sum_{i', j'} w(i, i', j, j', y) Y(-1, i, x + y', y' + y)$$

(l layer, j out-chann, i in-chann, K_l kernel); $Y = f(z)$; final softmax over Y ($L_1, \dots$).

Distributed: a 9×9 scan $\Rightarrow$ two stacked 3×3 ; layers build a hierarchy small $\rightarrow$ intermediate $\rightarrow$ full pattern $\Rightarrow$ lower-dim weights, fewer params, more generalizable.

Param-count example $K=16, N_1=4, N_2=2, N_3=1$ ($L_1=L_2=2$): non-distributed one 16×16 scan computed by $K^2 N_3 \approx 1034$ w; distributed $L_1^2 N_1 + L_2^2 N_1 + L_3^2 N_2 \approx 160$ w ($\approx 6.5\times$ fewer; deeper $\Rightarrow$ more savings).

Conv hyperparams • pooling

Filter / receptive field / stride S : filter scans prev map; receptive field = corresponding input patch (grows with depth); larger S = smaller map.

Deriv output size $\lfloor \frac{w}{s} \rfloor + 1$: filter (size M) top-left starts at 0; last valid start is $N-M$; with hop s the starts are $0, s, 2s, \dots \leq N-M$, i.e. $\lfloor \frac{N-M}{s} \rfloor + 1$ positions (+1 counts position 0). At $S=1$ conv shrinks map by $M-1$.

Zero-pad: pad 0 around $\Rightarrow$ output $\approx \lfloor N/S \rfloor$; "same" (map=inp) at $S=1$, enables arbitrarily many layers.

Max-pool: 1-px jitter shifts much of the map; replace each value by max over a neighborhood; usually non-overlapping (stride=size) $\Rightarrow$ "if any pattern in region, detected".

Max-pool backprop $\rightarrow$ **argmax**: from subgrad of max, $\frac{\partial \max_i x_i}{\partial x_i} = \frac{1}{x_i - \max_{j \neq i} x_j}$ (gradient routes only to the max element (others 0)).

Variants: avg-pool = mean (soft-max) $\Rightarrow$ grad spreads to all (more informative); in-chans $\frac{1}{L} \text{grad}$ (3/RGB); **FCN** – replace pool with strided conv ($S > 1$) = learned pooling.

Architectures

CNN = Conv layers + Max-pool + final MLP(softmax). 1D version = TDNN / Temporal Conv Net.

LeNet-5 (LeCun'98, digits, 32×32): $C1 \ 6@28^2 \Rightarrow S2 \ 6@14^2 \Rightarrow C3 \ 16@10^2 \Rightarrow S4 \ 16@5^2 \Rightarrow C5 \ 120 \Rightarrow F6 \ 84 \Rightarrow 10$. Alternating Conv/Subsample, then full-connect.

AlexNet (Krizhevsky-Sutskever-Hinton'12, ImageNet): 5 conv + 3 FC, max-pool, ReLU, 1000-way softmax $\Rightarrow$ state of DL era.

backprop 3 recurrences; softmax-CE logit grad $f_c = 1$ [c=y]; **conv output** $\frac{\partial L}{\partial z} \frac{\partial z}{\partial y} + 1$; **max-pool grad** $\rightarrow$ **argmax**; **tied-grad** $= \sum_{i \in S} \nabla_i z_i$; **1034 $\rightarrow$ 160 param saving**.

L03 Supervised Learning II – optimization normalization • architectures

Setup: min $L(\theta) = \frac{1}{N} \sum \text{err}(f(x_i; \theta), y_i)$ (non-convex). GD $X_{k+1} = X_k - \eta_k \nabla f(X_k)$; stop $f(X_{k+1}) - f(X_k) < \epsilon$. Convex theory below sets η .

1 • Convexity & smoothness

Convex: $f(\frac{x+y}{2}) \leq \frac{1}{2}(f(x) + f(y))$, equiv $\nabla^2 f \geq 0$; global min $\Rightarrow \nabla f = 0$.

Lipschitz: g is L -Lip if $|g(x) - g(y)| \leq L \|x - y\|$.

L -smooth: convex & gradient L -Lip: $\|\nabla f(y) - \nabla f(y')\| \leq L \|y - y'\|$; equiv $\nabla^2 f \leq LI$ (grad can't change too fast; weak, holds for NN). (ReLU has a kink; ELU in 0 处光滑, Lipschitz 更好)

strong-convx: $f(y) \geq f(x) + \nabla f(x)^T (y - x) + \frac{\mu}{2} \|y - x\|^2$, equiv $\nabla^2 f \geq \mu I > 0$ (quadratic lower bound).

2 • Descent lemma $\rightarrow$ optimal LR

Lemma (L -smooth): convex quadratic upper bound

$$f(y) \leq f(x) + \nabla f(x)^T (y - x) + \frac{L}{2} \|y - x\|^2$$

Deriv (best step): set $y = x - \eta \nabla f$, plug in & minimize over η :

$$f(y) \leq f(x) - \eta \|\nabla f\|^2 + \frac{L\eta^2}{2} \|\nabla f\|^2$$

$$\Rightarrow \left[\frac{\partial}{\partial \eta} \right] = 0 \quad (\text{optimal}); \text{ any } 0 < \eta < 2/L \text{ decreases}$$

Deriv (rate, $\eta=1/L$): sub $\eta=1/L$ above:

$$f(X_{k+1}) \leq f(X_k) - \frac{1}{2L} \|\nabla f(X_k)\|^2$$

$$\Rightarrow \|\nabla f(X_k)\|^2 \leq 2L(f(X_k) - f(X_{k+1}))$$

$$\text{sum } k=0..K \text{ (telescope, } f \geq f^*) \Rightarrow K \min \|\nabla f(X_k)\|^2 \leq 2L(f(X_0) - f^*)$$

$$\Rightarrow \min \|\nabla f\|^2 \leq \epsilon \text{ needs } K = O(1/\epsilon)$$

Sublinear $O(1/\epsilon)$; also holds **non-convex** $\rightarrow$ (stationary pt).

Strong-convx: $+ \mu$ gives contraction $f(X_{k+1}) - f^* \leq (1 - \frac{\mu}{L})(f(X_k) - f^*) \rightarrow$ **linear** $K = O(\log \frac{1}{\epsilon})$, $\kappa = L/\mu$.

Est L : backtracking/Armijo ($f(w - \eta \nabla f) \leq f(w) - \eta \|\nabla f\|^2$, $\gamma \in (0, \frac{1}{2})$); else shrink $\eta_i \rightarrow \alpha \eta$ val plateau. δ_i adds $\alpha \|\nabla f\|^2 \Rightarrow$ convex becomes μ -strong (better conditioning).

3 • Newton (2nd order)

Why: scalar η on all coords is wrong – each axis of $f = \frac{1}{2} x^T A x + b x + c$ (A=0) wants own LR.

Deriv (axis-scaling): change vars $X = A^{-1/2} \tilde{X} \Rightarrow f$ isotropic in $\tilde{X}$; plain GD $X \rightarrow \nabla f_X f$, map back ($X = A^{-1/2} \tilde{X}$, $\nabla_X = A^{-1/2} \nabla_{\tilde{X}}$).

$$\tilde{X}_{k+1} = \tilde{X}_k - \eta \nabla_{\tilde{X}} f$$

$$A^{1/2} \tilde{X}_{k+1} = A^{1/2} \tilde{X}_k - \eta A^{-1/2} \nabla_X f$$

$$\Rightarrow X_{k+1} = X_k - \eta A^{-1} \nabla_X f$$

General: replace A by Hessian $H = \nabla^2 f(X$

"Who won 2026 world cup?": pretrained = obs (.3/.4/.3); SFT target "I don't know"
loss (.2/.2/.6) **positives only**; RLHF rewards
Spain; **-4**, Argentina: **-4** = collapse to
IDK (.01/.02/.97). Caveat: always latest

the LLM doesn't know → motivates inference-
 nement. ► **3-stage RLHF pipeline: RM**

Reasoning Model: an LLM that "thinks"

- responding** - allow thinking via **longer** intermediate **thinking tokens hidden**
- time scaling**: more test-time compute → (o1 AIME pass@1 rises on log-scale)

Reasoning: CoT prompting (Google prompt → thinking → output). **Training:** = get the best thinking tokens. **Not** (no "correct" thinking tokens; we only get answers) → use **RL** to self-explore. Remark: exploration space huge → reward **ACCURATE**. **Reasoning:** Env = prompts needing long math/code/logic; Action = **thinking**.

output tokens; Reward = binary ± 1
 ct, -1 wrong) vs ground-truth **answer**
 loop: prompt $\rightarrow$ thinking+candidate (eg
 "4,4,7,7" $\rightarrow$ (4+4) $\times$ (7+7) $\rightarrow$ verifier $\rightarrow$
 update. Post-train scaling: **longer RL train**
 er (mean response length $\times$ reward both

→ language-only models know **facts** but images/video/audio/charts → lack perception. Cast vision tasks as **text on conditioned on visual input**.

VLM): input = text interleaved w/ videos, output = free-form text, supports **in-vision-shot**. Arch = two **frozen** pretrained (vision encoder + LM) + two additions:

- **Resampler** (compress many visual small fixed set of visual tokens) and
- **ATTN-DENSE** (cross-attn language→vision frozen LM, gated → starts as identity).

One model, many tasks via few-shot; **32**

Instruction tuning: few-shot $\neq$ in-following $\rightarrow$ want image chat. Arch = n enc + Vicuna LM + simple **projection** into LLM embedding space, instruction $\rightarrow$ response X_u . **Two stages:** (1) feature — train **only projection**; (2) end-to-end **joint enc, update projection+LLM**. arch GPTQ; eq. tuning 21.5% $\rightarrow$ full data 85.1%.

Unified: one model for understanding + editing + reasoning. **Decoder-only**: (understanding + generation) only. **multimodal self-attention** (all tokens cross both); **two visual pathways** — **ViT** (standing, VAE for generation; outputs on prediction + velocity prediction. Train **saved** text/image/video/web (not just

Analysis & Exam Recap

the family — what it learns / how it

- **Learning:** learn velocity u_t by CFM; sample $=$

σ ; DDPM=VP-SDE, NCSN=VE-SDE;
 FM.
es
 $\log^2$ (asym); JSD sym $\leq \log 2$ (GAN);
 $\|x-y\|$ (WGAN); Fisher $\mathbb{E}\|s_\theta - \nabla \log p\|^2$ (score);
 $\text{KL}(p_{\text{data}} \| p_\theta)$.
 ations to reproduce

- recurrences · softmax-CE $f_c - 1/c = y$ ·
- mma $\eta=1/L$ · Adam bias-correct · BN
- $\sigma > 0$ · stoch-flip $\rightarrow \sigma(z_i)$ · EBM pos-neg ($\frac{1}{\sigma} \nabla E$) · MH Z-cancel.
- $\frac{1}{\sigma} \rightarrow C(G) = 2\text{JSD} - \log 4$ · WGAN-GP.
- inal $\rightarrow$ posterior $\rightarrow$ ELBO $\rightarrow L_{\text{simple}}$ · ISM int-by-
- MM $\leftarrow \epsilon_{\text{p}}$ · Folkler-Planck · CFM = FM + c

α = 0.999; DCGAN 2e-4 β 1.5; clip 5; He $\mathcal{N}(0, 2/n_k)$;
 1000, β : $10^{-4} \rightarrow .02$, EMA .9999, attn@16;
 .01 n_c 5; GP λ 10; NCSN λ = σ^2 ; scaling
 .095, α_c .050; MHA d_k = d/h .

Index – Gaussian essentials

Integral (the one in Gaussian-KL)

$$\int a^T M(x-a) dx = \text{Tr}(M\Sigma) + (\mu-a)^T M(\mu-a) \quad (\text{and } \int dx=1).$$